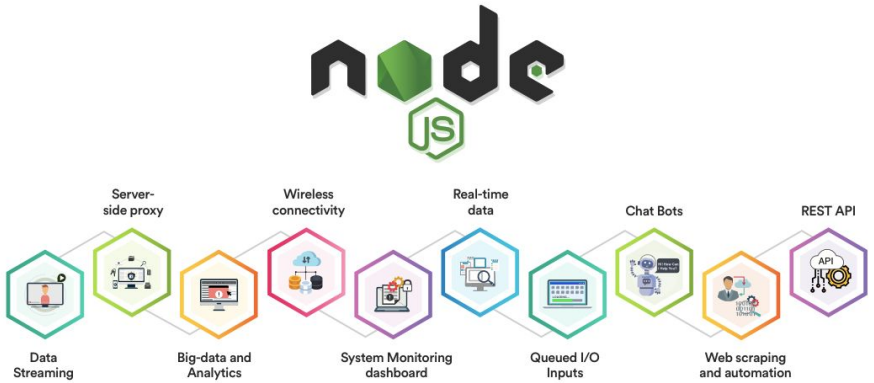
Servidores - NodeJS

Introducción

Node JS

- Creado en 2009 - Ryan Dahl
- Server-Side Javascript
  - V8 JavaScript engine
- Single Process
  - Single Thread for every request
  - Non-Blocking paradigm
- Gran comunidad
- Variedad de Librerías y Frameworks



NodeJS vs Browser

- No **document**, **window**, ... objects
- Saber la versión exacta en la que va a correr el código **vs** no saber en qué navegador va a correr
- Accesso a APIs muy potentes como **fs**
- **CommonJS** y **ES** module systems

Primer Servidor

```
1  const http = require('http');
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

Primer Servidor

```
1   const http = require('http');
2
3
4
5
6
7
8
9
10
11  const server = http.createServer(listener);
12
13
14
15
```

Primer Servidor

```
1   const http = require('http');
2
3   const host = 'localhost';
4   const port = 3030;
5
6
7
8
9
10
11  const server = http.createServer(listener);
12
13  server.listen(port, host, () => {
14    console.log(`Servidor levantado en http://${host}:${port}`)
15  })
```

Primer Servidor

```
1   const http = require('http');
2
3   const host = 'localhost';
4   const port = 3030;
5
6   const listener = (req, res) => {
7     res.writeHead(200);
8     res.end('Node es genial!');
9   }
10
11  const server = http.createServer(listener);
12
13  server.listen(port, host, () => {
14    console.log(`Servidor levantado en http://${host}:${port}`)
15  })
```

Primer Servidor

```
1  const http = require('http');
2
3  const host = 'localhost';
4  const port = 3030;
5
6  const listener = (req, res) => {
7    res.writeHead(200);
8    res.end('Node es genial!');
9  }
10
11 const server = http.createServer(listener);
12
13 server.listen(port, host, () => {
14   console.log(`Servidor levantado en http://${host}:${port}`)
15 })
```

La función “listener” se va a ejecutar por cada request

Muchos Clientes -> 1 servidor

NodeJS Server

```
const http = require('http');

const host = 'localhost';
const port = 3030;

const listener = (req, res) => {
  res.writeHead(200);
  res.end('Node es genial!');
}

const server = http.createServer(listener);

server.listen(port, host, () => {
  console.log(`Servidor levantado en http://${host}:${port}`)
})
```

HTTP Request 1

NodeJS Server

```
const http = require('http');

const host = 'localhost';
const port = 3030;

const listener = (req, res) => {
  res.writeHead(200);
  res.end('Node es genial!');
}

const server = http.createServer(listener);

server.listen(port, host, () => {
  console.log(`Servidor levantado en http://${host}:${port}`)
})
```

HTTP Request 1

NodeJS Server

```
const http = require('http');

const host = 'localhost';
const port = 3030;

const listener = (req, res) => {
  res.writeHead(200);
  res.end('Node es genial!');
}

const server = http.createServer(listener);

server.listen(port, host, () => {
  console.log(`Servidor levantado en http://${host}:${port}`)
})
```

HTTP Response 1

NodeJS Server

```
const http = require('http');

const host = 'localhost';
const port = 3030;

const listener = (req, res) => {
  res.writeHead(200);
  res.end('Node es genial!');
}

const server = http.createServer(listener);

server.listen(port, host, () => {
  console.log(`Servidor levantado en http://${host}:${port}`)
})
```

“Node es genial!”

HTTP Response 1

NodeJS Server

```
const http = require('http');

const host = 'localhost';
const port = 3030;

const listener = (req, res) => {
  res.writeHead(200);
  res.end('Node es genial!');
}

const server = http.createServer(listener);

server.listen(port, host, () => {
  console.log(`Servidor levantado en http://${host}:${port}`)
})
```

HTTP Request 1

HTTP Request 2

HTTP Request 3

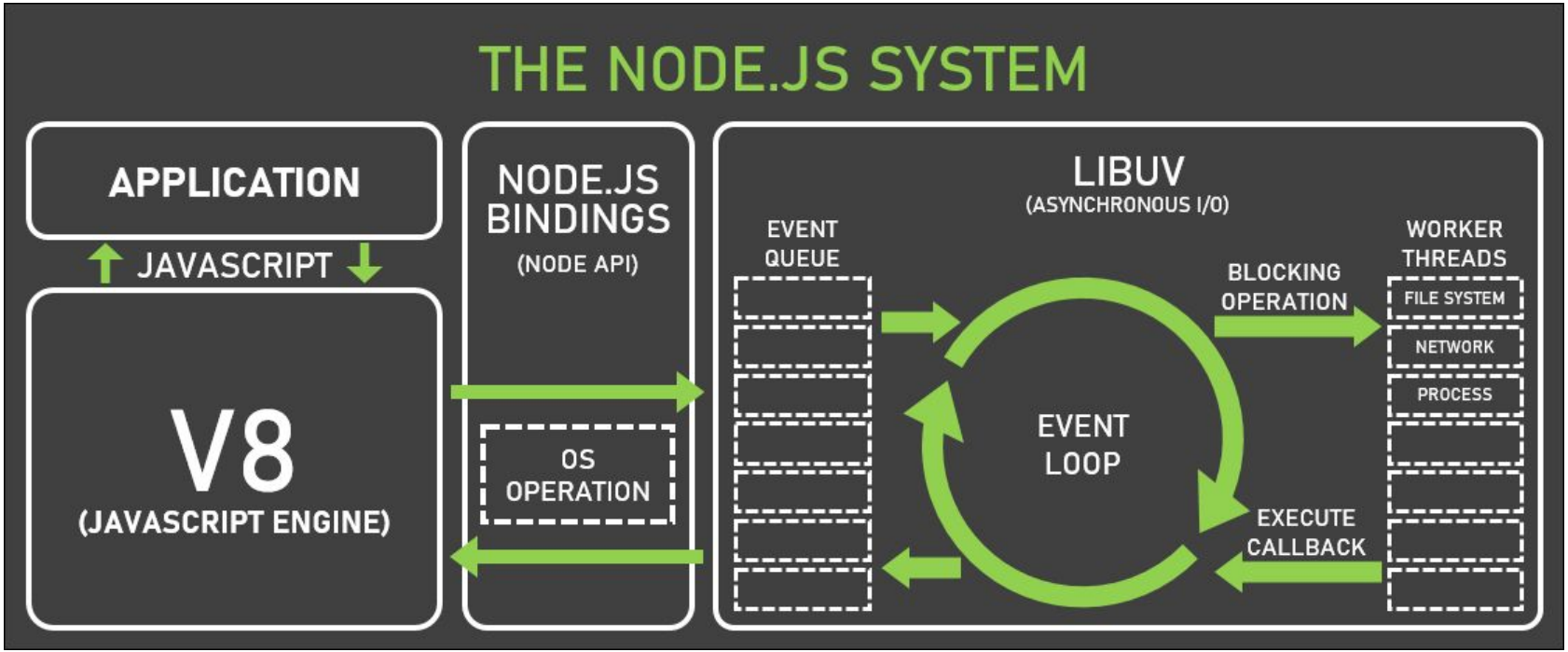
HTTP Request 4

1 Solo Hilo…. muchos requests…

NodeJS Server

```
const http = require('http');

const host = 'localhost';
const port = 3030;

const listener = (req, res) => {
  res.writeHead(200);
  res.end('Node es genial!');
}

const server = http.createServer(listener);

server.listen(port, host, () => {
  console.log(`Servidor levantado en http://${host}:${port}`)
})
```

Node JS - Event Loop



Analicemos el código de nuevo...

```
1   const http = require('http');
2
3   const host = 'localhost';
4   const port = 3030;
5
6   const listener = (req, res) => {
7     res.writeHead(200);
8     res.end('Node es genial!');
9   }
10
11  const server = http.createServer(listener);
12
13  server.listen(port, host, () => {
14    console.log(`Servidor levantado en http://${host}:${port}`)
15  })
```

Analicemos el código de nuevo...

```
1   const http = require('http');
2
3   const host = 'localhost';
4   const port = 3030;
5
6   const listener = (req, res) => {
7     res.writeHead(200);
8     res.end('Node es genial!');
9   }
10
11  const server = http.createServer(listener);
12
13  server.listen(port, host, () => {
14    console.log(`Servidor levantado en http://${host}:${port}`)
15  })
```

Analicemos el código de nuevo...

```
1   const http = require('http');
2
3   const host = 'localhost';
4   const port = 3030;
5
6   const listener = (req, res) => {
7     res.writeHead(200);
8     res.end('Node es genial!');
9   }
10
11  const server = http.createServer(listener);
12
13  server.listen(port, host, () => {
14    console.log(`Servidor levantado en http://${host}:${port}`)
15  })
```

Req: IncomingMessage

IncomingMessage

destroy()	
headers	Returns a key-value pair object containing header names and values
httpVersion	Returns the HTTP version sent by the client
method	Returns the request method
rawHeaders	Returns an array of the request headers
rawTrailers	Returns an array of the raw request trailer keys and values
setTimeout()	Calls a specified function after a specified number of milliseconds
statusCode	Returns the HTTP response status code
socket	Returns the Socket object for the connection
trailers	Returns an object containing the trailers
url	Returns the request URL string

ServerResponse

addTrailers()	Adds HTTP trailing headers
end()	Signals that the the server should consider that the response is complete
finished	Returns true if the response is complete, otherwise false
getHeader()	Returns the value of the specified header
headersSent	Returns true if headers were sent, otherwise false
removeHeader()	Removes the specified header
sendDate	Set to false if the Date header should not be sent in the response. Default true
setHeader()	Sets the specified header
setTimeout	Sets the timeout value of the socket to the specified number of milliseconds
statusCode	Sets the status code that will be sent to the client
statusMessage	Sets the status message that will be sent to the client
write()	Sends text, or a text stream, to the client
writeContinue()	Sends a HTTP Continue message to the client
writeHead()	Sends status and response headers to the client

NodeJS Streams

Streams

Readable Streams	Writable Streams
HTTP responses, on the client	HTTP requests, on the client
HTTP requests, on the server	HTTP responses, on the server
fs read streams	fs write streams
zlib streams	zlib streams
crypto streams	crypto streams
TCP sockets	TCP sockets
child process stdout and stderr	child process stdin
process.stdin	process.stdout, process.stderr

NodeJS Streams

Readable Streams

Events

- data
- end
- error
- close
- readable

Functions

- pipe(), unpipe()
- read(), unshift(), resume()
- pause(), isPaused()
- setEncoding()

Writable Streams

Events

- drain
- finish
- error
- close
- pipe/unpipe

Functions

- write()
- end()
- cork(), uncork()
- setDefaultEncoding()

NodeJS Streams

```
const fs = require('fs');
const archivo = fs.createWriteStream('./big.file');

for(let i=0; i<= 1e6; i++) {
  file.write('Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusm
}

file.end();
```

NodeJS Streams

```
const fs = require('fs');
const servidor = require('http').createServer();

servidor.on('request', (peticion, respuesta) => {
  fs.readFile('./big.file', (error, data) => {
    if (error) throw error;

    respuesta.end(data);
  });
});

servidor.listen(8000);
```

NodeJS Streams

```
const fs = require('fs');
const servidor = require('http').createServer();

servidor.on('request', (peticion, respuesta) => {
  fs.readFile('./big.file', (error, data) => {
    if (error) throw error;

    respuesta.end(data);
  });
});

servidor.listen(8000);
```

NodeJS Streams

```
const fs = require('fs');
const servidor = require('http').createServer();

servidor.on('request', (peticion, respuesta) => {
  const src = fs.createReadStream('./big.file');
  src.pipe(respuesta);
});

servidor.listen(8000);
```

Eficiente en Memoria!!

NodeJS Streams

```
a.pipe(b).pipe(c).pipe(d)

# Que es equivalente a:
a.pipe(b)
b.pipe(c)
c.pipe(d)

# Que, en Linux, es equivalente a:
$ a | b | c | d
```

NodeJS Streams

```
# legible.pipe(escribible)

legible.on('data', (trozo) => {
  escribible.write(trozo);
});

legible.on('end', () => {
  escribible.end();
});
```

RESTful APIs

Estilo arquitectónico REST

- “representational state transfer “
- **Restricciones arquitectónicas**
  - La información es brindada en diferentes formatos:
    - **JSON es el más utilizado**
- Stateless client-server communication
- Operaciones sobre **recursos**:
  - POST -> crear
  - PUT -> actualizar
  - GET
  - DELETE

RESTful APIs - Ejemplos

GET /api/usuarios -> Retorna los usuarios del sistema

POST /api/usuarios -> Crea un usuario

PUT /api/usuarios/1234 -> Actualiza un usuario con id = “1234”

GET /api/usuarios/1234 -> Retorna el usuario con id = “1234”

RESTful APIs - Buenas Prácticas

- Usar **sustantivos** en vez de **verbos** en los endpoints
- Anidar recursos si son lógicamente dependientes
  - GET /api/articulos/1234/comentarios -> Retorna los comentarios del articulo 1234
- Retornar errores estandarizados
  - 400 Bad Request
  - 401 Unauthorized
  - 403 Forbidden
  - 404 Not Found
  - 500 Internal server error
- Permitir filtros, ordenamiento y paginación
  - GET /api/usuarios?edad=20&sort=nombre

Fuentes

- https://nodejs.org/learn/getting-started/introduction-to-nodejs
- https://nodejs.org/api/stream.html
- https://nodejs.org/learn/modules/how-to-use-streams
- https://www.freecodecamp.org/espanol/news/node-js-streams-todo-lo-que-necesitas-saber/
- https://nodejs.org/learn/modules/how-to-use-streams
- https://www.w3schools.com/nodejs/obj_http_incomingmessage.asp
- https://nodejs.org/learn/getting-started/differences-between-nodejs-and-the-browser#differences-between-nodejs-and-the-browser
-